

Proyecto final

Presentación ejecutiva

Presentado por:

Edgar Andrés Romero Otálora
Julian David Miranda Leguizamón
Sebastian Bedoya Florez
Jhon Alexander Tenjo Romero

Contexto

Ecommify es un marketplace que gestiona todo el ciclo de comercio electrónico (productos, órdenes, pagos, logística y reseñas). Su principal desafío es equilibrar dos cargas opuestas: transacciones críticas que requieren consistencia estricta (órdenes y pagos) y consultas masivas de baja latencia (catálogo, reseñas y geolocalización).

Un solo motor de base de datos no puede optimizar ambas sin compromisos en rendimiento o integridad. Por ello, la solución propuesta es una arquitectura híbrida, donde PostgreSQL maneja las transacciones críticas (Write Model) y MongoDB las consultas y lecturas escalables (Read Model), sincronizados de forma asíncrona mediante eventos.

Objetivos

Diseñar y evaluar una arquitectura híbrida de base de datos para Ecommify usando PostgreSQL y MongoDB, que asegure consistencia en transacciones críticas y alto rendimiento en consultas masivas, basada en el Teorema CAP y evidencia de rendimiento.

Objetivos específicos:

- Modelar en PostgreSQL las entidades transaccionales con integridad ACID.
- Diseñar en MongoDB estructuras optimizadas para lectura masiva.
- Justificar decisiones bajo el Teorema CAP y sus trade-offs.
- Proponer estrategias de escalamiento y migración a producción.

Arquitectura Híbrida

Modelo híbrido: separación de datos transaccionales y analíticos con un bus de eventos.

PostgreSQL (transaccional / ACID):

Órdenes, pagos e inventario

Integridad y consistencia fuerte

MongoDB (analítico / lectura):

Catálogo, reseñas y geolocalización

Consultas rápidas y flexibles

Optimización:

Duplicación controlada de datos (extended reference)

Pre-cálculo de métricas (computed pattern)

Índices geoespaciales (2dsphere)

Principales decisiones técnicas y justificaciones

- **PostgreSQL para datos transaccionales (ACID)**

- Garantiza **consistencia e integridad** en clientes, órdenes, pagos e inventario
- Evita duplicidad o pérdida de información en procesos críticos
- Soporta relaciones complejas entre entidades

- **MongoDB para datos flexibles y de alto volumen**

- Maneja **estructuras variables** (productos, reseñas, geolocalización)
- Optimizado para **lecturas masivas y escalabilidad horizontal (sharding)**
- Alta disponibilidad en escenarios de partición

- **Diseño orientado a CAP**

- PostgreSQL → prioriza Consistencia (C)
- MongoDB → prioriza Disponibilidad y Partición (A + P) según dominio

- **Optimización por dominio**

- Geolocalización: consultas rápidas para costos de envío
- Reviews: crecimiento continuo y acceso frecuente
- Catálogo: alta concurrencia de lectura

Resultados de evaluación de rendimiento

PostgreSQL

A partir de EXPLAIN ANALYZE, se realizaron consultas antes y después de optimizar las tablas y se obtuvo lo siguiente:

	Consulta	Plan de ejecución antes	Antes (ms)	Plan de ejecución despues	Después (ms)	Mejora (%)
0	Búsqueda de categorías	Seq Scan	5.473	Seq Scan	0.068	98.757537
1	Búsqueda de ciudades	Seq Scan	86.503	Bitmap Heap Scan y Bitmap Or	3.041	96.484515
2	Historial de compras de un cliente	Seq Scan y hash join	2307.889	Index Scan	1323.246	42.664227
3	Reporte de ventas totales	Seq Scan, hash join y sort	225.461	Seq Scan, hash join y sort	265.891	-17.932148
4	Reporte de ventas canceladas	Seq Scan, hash join y sort	12.978	Index Scan, Seq Scan, hash join y sort	6.231	51.987980

Resultados de evaluación de rendimiento

MongoDB

Se evaluó la carga concurrente (1 a 50 usuarios) y la escalabilidad del pipeline de geolocalización ya particionado, filtrando por % de estados de Brasil.

Usuarios	Peticiones	Throughput (q/s)	Latencia prom. (ms)	Latencia P95 (ms)	Latencia máx. (ms)	% estados	N° estados	Tiempo total (ms)	execution TimeMillis
1	3	18.30	54.32	45.55	79.89	25%	6	49.90	19
5	15	71.55	63.55	155.78	158.94	50%	13	51.27	23
10	30	144.23	60.09	170.99	196.64	75%	20	48.99	25
25	75	50.29	299.36	1448.94	1479.69	100%	27	51.02	26
50	150	75.14	457.27	1951.06	1976.45				

*Docs examinados: 19,015
(constante en las 4 pruebas)*

Análisis comparativo PostgreSQL vs MongoDB

Aspecto	Motor escogido	Justificación técnica
ACID y Transaccionalidad	PostgreSQL	Asegura aislamiento, integridad referencial y control de concurrencia. Evita anomalías críticas como dobles cobros o sobreventa en órdenes y pagos.
Read-Heavy (consultas intensivas)	MongoDB	Evita JOINs costosos usando BSON y documentos embebidos. Mejora latencia con patrones como <i>Computed Pattern</i> y consultas desde índices.
Estructuras heterogéneas	MongoDB	Soporta esquemas polimórficos, permitiendo productos con atributos variables sin complejidad relacional.
Procesamiento geoespacial	MongoDB	Usa índices 2dsphere y operadores \$near / \$geoWithin. Permite análisis masivo optimizado (6,349 registros, ~11 ms respuesta).
Integridad y reglas de dominio	PostgreSQL	Define reglas estrictas a nivel de esquema, asegurando consistencia en datos financieros y logísticos antes de cambios de estado.
Escalabilidad horizontal	MongoDB	Sharding con hashed shard key permite distribución lineal de carga. PostgreSQL requiere replicación o particionamiento más complejo.

Análisis del Teorema CAP aplicado

Colección	Estructura	Volumen y Velocidad	Teorema CAP
customers	Estructura relacional. Clientes con ciudad, estado y código postal. Relación con orders.	~99k registros. Lectura/escritura media por consultas frecuentes.	(C) Consistencia para integridad de clientes y pedidos.
orders	Tabla transaccional central con relaciones a customers, payments y order_items.	~99k pedidos. Alta carga transaccional.	(C) Evita duplicidad o pérdida de pedidos.
order_items	Relación entre órdenes, productos y vendedores.	~112k registros. Alta inserción y análisis.	(C) Garantiza integridad entre entidades.

Análisis del Teorema CAP aplicado

Colección	Estructura	Volumen y Velocidad	Teorema CAP
order_payments	Datos financieros: tipo de pago, montos, cuotas.	~103k registros. Escrituras moderadas y lecturas frecuentes.	(C) Consistencia en pagos y facturación.
sellers	Información de vendedores con ubicación geográfica.	~3k registros. Lecturas frecuentes, pocas actualizaciones.	(C) Integridad en referencias de ventas y logística.
categories	Traducción de categorías (PT → EN).	71 registros. Baja frecuencia de uso.	(CA) Alta consistencia y disponibilidad por su tamaño reducido.

Análisis del Teorema CAP aplicado

Colección	Estructura	Volumen y Velocidad	Teorema CAP
products	Estructura polimórfica según categoría (atributos variables).	~32k registros. Lecturas extremadamente altas.	(P, A) Resiste particiones y mantiene disponibilidad del catálogo.
order_reviews	Documentos semi-estructurados por product_id.	+100k registros. Alta lectura, escritura media.	(P, A) Operación continua incluso con fallos de red.
geolocation	GeoJSON con coordenadas [long, lat].	+1M registros. Consultas geoespaciales rápidas.	(P, A) Sharding + respuesta en milisegundos para envíos.

Lecciones aprendidas y desafíos superados

La consistencia eventual es un diseño, no un default

Requiere Outbox + Kafka + idempotencia + DLQ para evitar datos divergentes

La complejidad está en las fronteras entre motores

No depende del volumen de datos, sino del flujo de sincronización entre sistemas.

La arquitectura mejora la resiliencia del sistema

Fallas en un motor no afectan el otro → aislamiento de impacto.

La división de motores depende de garantías, no del tipo de dato

PostgreSQL para consistencia fuerte / MongoDB para disponibilidad y velocidad.

Cada motor se optimiza distinto y no es comparable directamente

Se evalúa por rol (write model vs read model), no por métricas aisladas.

El modelado de datos es más determinante que el motor

Un buen diseño puede generar más impacto que el cambio de tecnología.

Recomendaciones estratégicas

Estado actual y crecimiento 10x

Colección	Motor	Registros actuales	Crecimiento esperado	Riesgo de degradación
geolocation	MongoDB	1.000.163	Muy alto	Crítico
order_items	PostgreSQL	112.650	Muy alto	Alto
customers	PostgreSQL	99.441	Alto	Medio
orders	PostgreSQL	99.441	Alto	Medio
order_payments	PostgreSQL	103.886	Alto	Medio
order_reviews	MongoDB	99.224	Medio	Bajo / Medio
products	MongoDB	32.951	Medio	Bajo
sellers	PostgreSQL	3.095	Bajo	Bajo
categories	PostgreSQL	71	Bajo	Bajo
zip_codes	PostgreSQL	15.078	Bajo	Bajo

Recomendaciones estratégicas

Disparadores de escalamiento

Entidad	Umbral volumen	Umbral latencia	Acción
geolocation	1.500.000 docs	>200 ms	Sharding por zona geográfica
order_items	300.000 filas	p95 > 150 ms	Particionamiento por fecha
orders	250.000 filas	p95 > 100 ms	Read réplica PostgreSQL
order_payments	250.000 filas	>300 ms	Índice parcial por estado
products	100.000 docs	>500 ms	Elasticsearch

Recomendaciones estratégicas

Estrategias técnicas clave

Estrategia	Descripción
Sharding geográfico	geolocation se distribuye por estado y código postal (zone sharding)
Particionamiento	order_items dividido por trimestre para reducir costo de consultas 60–80%
Read réplica	Desacople de lectura/escritura en customers, orders y payments

Recomendaciones estratégicas

Fases de escalamiento

Fase	Volumen aprox	Acciones principales
Fase 1: Optimización	Hasta 2x	Índices compuestos, deduplicación, PgBouncer, monitoreo
Fase 2: Separación de cargas	2x – 5x	Read réplica, Redis cache, particionamiento
Fase 3: Escalamiento horizontal	5x – 10x	Sharding, Elasticsearch, distribución de carga
Fase 4: Alta disponibilidad	10x+	Multi-región, failover, réplica sets en MongoDB

Recomendaciones estratégicas

Limitaciones del entorno actual

Componente actual	Limitación crítica	Impacto en producción
Supabase Free Tier	500 MB almacenamiento, 2 conexiones	Insuficiente para escala real
Google Colab	Sin persistencia, sesiones temporales	Inviabile para producción
MongoDB	Sin backups automáticos	Riesgo crítico de pérdida de datos
Sin Kafka	Sin Outbox / CDC implementado	Sin sincronización robusta entre modelos
Sin observabilidad	No hay métricas de rendimiento	No se puede detectar degradación

Dimensionamiento de infraestructura (Fase 1)

Componente	Especificación recomendada	Costo mensual estimado
PostgreSQL gestionado (AWS RDS)	2 vCPU, 8 GB RAM	\$110 USD
Read réplica PostgreSQL	2 vCPU, 8 GB RAM	\$65 USD
MongoDB Atlas M10 (3 nodos)	2 vCPU, 2 GB RAM, 10 GB storage	\$57 USD
Redis (ElastiCache)	1 vCPU, 0.5 GB RAM	\$12 USD
Kafka gestionado (MSK)	Para Outbox / CDC	\$30 – \$60 USD
Instancia de aplicación	2 vCPU, 2 GB RAM	\$15 USD
Almacenamiento + transferencia	50 GB + 100 GB egress	\$20 USD

Fase	Descripción	Costo mensual
Fase 1	Infraestructura base sin Kafka	\$279 USD / mes
Fase 2	Con Kafka + CDC activo	\$329 – \$359 USD / mes

Recomendaciones estratégicas

Tecnologías complementarias recomendadas

Redis (caché distribuido)

Reduce carga directa a PostgreSQL y MongoDB en lecturas frecuentes.

- Sesiones de usuario (TTL 30 min)
- Catálogo de productos (TTL 10 min)
- Búsquedas frecuentes (TTL 5 min)
- Carrito activo (TTL 24 h)
- Rate limiting API (TTL 1 min)
- Estado de órdenes (TTL 2 min)

Impacto: menor latencia + reducción de carga en DBs críticas.

Elasticsearch (motor de búsqueda)

Mejora búsqueda sobre catálogo en MongoDB.

- Full-text search con ranking (BM25)
- Autocompletado y tolerancia a errores
- Búsqueda facetada eficiente
- Escala mejor que \$regex / \$text

Arquitectura:

MongoDB (source of truth) → Kafka → Elasticsearch (índice de búsqueda)

Observabilidad (Prometheus + Grafana)

Métricas críticas del sistema:

- Index Efficiency Ratio (DB performance)
- p95/p99 query time (PostgreSQL)
- Replication lag (MongoDB)
- Cache hit ratio (Redis)
- Throughput geolocation / reviews
- Conexiones activas (PostgreSQL)
- Kafka consumer lag (futuro CDC)

Stack recomendado: Grafana Cloud (Prometheus + Loki + Tempo)

Conclusiones

Objetivos cumplidos con evidencia empírica

Esquema relacional en PostgreSQL y documental en MongoDB, implementados y validados con pruebas de carga reales.

El resultado más significativo: geolocation

Deduplicación + índice compuesto lograron 162x de aceleración, con Index Efficiency Ratio de 1.00.

Autocrítica principal del equipo

El mayor logro vino del modelado de datos, no del motor: depurar antes de modelar, modelar antes de elegir tecnología.

Arquitectura evaluada bajo el Teorema CAP

PostgreSQL en CA (consistencia) y MongoDB en AP (disponibilidad y partición), con trade-offs documentados por módulo.

Límites reales medidos con pruebas de carga

PostgreSQL no escala linealmente tras 20 usuarios; MongoDB cae 65% en throughput entre 10 y 25 usuarios.

Hoja de ruta abierta

Instrumentar lag de Kafka/réplicas, migrar a producción (~USD 279/mes en Fase 1) y adoptar Redis, Elasticsearch y observabilidad.